SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: **Database Management Systems**

COURSE CODE: **CSA05**

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Debugging & Optimisation Task (Low)

Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply

5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand
6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand
9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply
10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply

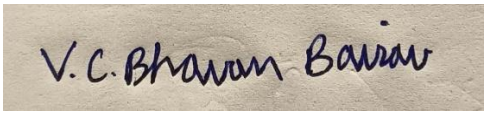
---

Code of Conduct:

I V.C.Bhavan Bairav - 192511369 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1) For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.

AIM:

To identify the functional dependencies and candidate key of the given relation.

GIVEN RELATION:

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE STUDENT_COURSE (
    ->     SID INT,
    ->     SName VARCHAR(30),
    ->     CID VARCHAR(10),
    ->     CName VARCHAR(30),
    ->     Instructor VARCHAR(30),
    ->     Grade VARCHAR(5),
    ->     PRIMARY KEY (SID, CID)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO STUDENT_COURSE VALUES
    -> (101, 'Arun',  'C101', 'DBMS',     'Dr.Ravi',   'A'),
    -> (101, 'Arun',  'C102', 'DSA',      'Dr.Meena',  'B'),
    -> (102, 'Meena', 'C101', 'DBMS',     'Dr.Ravi',   'A'),
    -> (103, 'Kiran', 'C103', 'Networks', 'Dr.Suresh', 'B');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM STUDENT_COURSE;
+-----+-------+------+----------+------------+-------+
| SID | SName | CID  | CName    | Instructor | Grade |
+-----+-------+------+----------+------------+-------+
| 101 | Arun  | C101 | DBMS     | Dr.Ravi    | A     |
| 101 | Arun  | C102 | DSA      | Dr.Meena   | B     |
| 102 | Meena | C101 | DBMS     | Dr.Ravi    | A     |
| 103 | Kiran | C103 | Networks | Dr.Suresh  | B     |
+-----+-------+------+----------+------------+-------+
4 rows in set (0.00 sec)

mysql>
```

QUERY 1 – Verify SID → SName

```
mysql> SELECT SID, COUNT(DISTINCT SName) AS Name_Count
    -> FROM STUDENT_COURSE
    -> GROUP BY SID;
+-----+------------+
| SID | Name_Count |
+-----+------------+
| 101 |          1 |
| 102 |          1 |
| 103 |          1 |
+-----+------------+
3 rows in set (0.00 sec)

mysql>
```

Therefore:

- SID → SName

QUERY 2 – Verify CID → CName, Instructor

```
mysql> SELECT CID,
    ->        COUNT(DISTINCT CName) AS Course_Count,
    ->        COUNT(DISTINCT Instructor) AS Instructor_Count
    -> FROM STUDENT_COURSE
    -> GROUP BY CID;
+------+--------------+------------------+
| CID  | Course_Count | Instructor_Count |
+------+--------------+------------------+
| C101 |            1 |                1 |
| C102 |            1 |                1 |
| C103 |            1 |                1 |
+------+--------------+------------------+
3 rows in set (0.00 sec)

mysql>
```

Therefore:

- CID → CName
- CID → Instructor

QUERY 3 – Verify Candidate Key (SID, CID)

```
mysql> SELECT SID, CID, COUNT(*) AS Record_Count
    -> FROM STUDENT_COURSE
    -> GROUP BY SID, CID;
+-----+------+--------------+
| SID | CID  | Record_Count |
+-----+------+--------------+
| 101 | C101 |            1 |
| 101 | C102 |            1 |
| 102 | C101 |            1 |
| 103 | C103 |            1 |
+-----+------+--------------+
4 rows in set (0.00 sec)

mysql>
```

Every (SID, CID) combination uniquely identifies one record.

Therefore:

- (SID, CID) → Grade
- Candidate Key = (SID, CID)

FUNCTIONAL DEPENDENCIES:

- SID → SName
- CID → CName
- CID → Instructor
- (SID, CID) → Grade

CANDIDATE KEY:

- (SID, CID)

RESULT:

Thus, the functional dependencies were identified and **(SID, CID)** was determined as the candidate key of the STUDENT_COURSE relation successfully.

2) Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

AIM:

To demonstrate the process of converting an unnormalized relation into First Normal Form (1NF) by removing repeating and multi-valued attributes.

UNNORMALIZED RELATION:

Consider a RESEARCHER_PROJECT relation where a researcher can work on multiple projects.

```
mysql> CREATE TABLE RESEARCHER_PROJECT_UNF (
    ->     ResearcherID VARCHAR(10),
    ->     ResearcherName VARCHAR(30),
    ->     Projects VARCHAR(100)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RESEARCHER_PROJECT_UNF VALUES
    -> ('R101', 'Arun', 'AI, Robotics'),
    -> ('R102', 'Meena', 'IoT'),
    -> ('R103', 'Kiran', 'Cyber Security, Cloud');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RESEARCHER_PROJECT_UNF;
+--------------+----------------+-----------------------+
| ResearcherID | ResearcherName | Projects              |
+--------------+----------------+-----------------------+
| R101         | Arun           | AI, Robotics          |
| R102         | Meena          | IoT                   |
| R103         | Kiran          | Cyber Security, Cloud |
+--------------+----------------+-----------------------+
3 rows in set (0.00 sec)

mysql>
```

PROBLEM:

- Projects = AI, Robotics
- Projects = Cyber Security, Cloud

These are non-atomic/multiple values**,** violating 1NF.

CONVERT TO 1NF:

Each project is placed in a **separate row**.

```
mysql> CREATE TABLE RESEARCHER_PROJECT_1NF (
    ->     ResearcherID VARCHAR(10),
    ->     ResearcherName VARCHAR(30),
    ->     Project VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RESEARCHER_PROJECT_1NF VALUES
    -> ('R101', 'Arun', 'AI'),
    -> ('R101', 'Arun', 'Robotics'),
    -> ('R102', 'Meena', 'IoT'),
    -> ('R103', 'Kiran', 'Cyber Security'),
    -> ('R103', 'Kiran', 'Cloud');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RESEARCHER_PROJECT_1NF;
+--------------+----------------+----------------+
| ResearcherID | ResearcherName | Project        |
+--------------+----------------+----------------+
| R101         | Arun           | AI             |
| R101         | Arun           | Robotics       |
| R102         | Meena          | IoT            |
| R103         | Kiran          | Cyber Security |
| R103         | Kiran          | Cloud          |
+--------------+----------------+----------------+
5 rows in set (0.00 sec)

mysql>
```

STEP-BY-STEP JUSTIFICATION:

Before 1NF:

- R101 → AI, Robotics
- R102 → IoT
- R103 → Cyber Security, Cloud

After 1NF:

- R101 → AI
- R101 → Robotics
- R102 → IoT
- R103 → Cyber Security
- R103 → Cloud

Now every attribute contains only one atomic value**.**

RESULT:

Thus, the unnormalized RESEARCHER_PROJECT_UNF relation was successfully converted into 1NF by removing the repeating/multi-valued Projects attribute and storing each project as a separate row.

3) Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.

AIM:

To analyze the given relation using functional dependencies and determine whether decomposition is required to achieve Second Normal Form (2NF).

GIVEN RELATION:

```
mysql> CREATE TABLE RESEARCH_RECORD (
    ->     A INT PRIMARY KEY,
    ->     B VARCHAR(20),
    ->     C VARCHAR(20),
    ->     D VARCHAR(20),
    ->     E VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RESEARCH_RECORD VALUES
    -> (101, 'B1', 'C1', 'D1', 'E1'),
    -> (102, 'B2', 'C2', 'D2', 'E2'),
    -> (103, 'B3', 'C3', 'D3', 'E3'),
    -> (104, 'B4', 'C4', 'D4', 'E4');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM RESEARCH_RECORD;
+-----+------+------+------+------+
| A   | B    | C    | D    | E    |
+-----+------+------+------+------+
| 101 | B1   | C1   | D1   | E1   |
| 102 | B2   | C2   | D2   | E2   |
| 103 | B3   | C3   | D3   | E3   |
| 104 | B4   | C4   | D4   | E4   |
+-----+------+------+------+------+
4 rows in set (0.00 sec)
```

FUNCTIONAL DEPENDENCIES

- A → BC
- BC → D
- D → E

QUERY – Verify Candidate Key A

```
mysql> CREATE TABLE RESEARCH_RECORD (
    ->     A INT PRIMARY KEY,
    ->     B VARCHAR(20),
    ->     C VARCHAR(20),
    ->     D VARCHAR(20),
    ->     E VARCHAR(20)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RESEARCH_RECORD VALUES
    -> (101, 'B1', 'C1', 'D1', 'E1'),
    -> (102, 'B2', 'C2', 'D2', 'E2'),
    -> (103, 'B3', 'C3', 'D3', 'E3'),
    -> (104, 'B4', 'C4', 'D4', 'E4');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT A,
    ->        COUNT(DISTINCT B) AS B_count,
    ->        COUNT(DISTINCT C) AS C_count,
    ->        COUNT(DISTINCT D) AS D_count,
    ->        COUNT(DISTINCT E) AS E_count
    -> FROM RESEARCH_RECORD
    -> GROUP BY A;
+-----+---------+---------+---------+---------+
| A   | B_count | C_count | D_count | E_count |
+-----+---------+---------+---------+---------+
| 101 |       1 |       1 |       1 |       1 |
| 102 |       1 |       1 |       1 |       1 |
| 103 |       1 |       1 |       1 |       1 |
| 104 |       1 |       1 |       1 |       1 |
+-----+---------+---------+---------+---------+
4 rows in set (0.00 sec)

mysql>
```

A+ = {A,B,C,D,E}

Therefore, Candidate Key = A.

CHECK FOR PARTIAL DEPENDENCY

A partial dependency occurs when a non-key attribute depends on part of a composite candidate key**.**

Here:

- Candidate Key = A

The key contains only one attribute.

Therefore, a partial dependency cannot exist. Hence the table is already in 2NF therefore No decomposition required

RESULT:

Thus, the given relation R(A,B,C,D,E) is already in 2NF, since its candidate key is the single attribute A and therefore no partial dependency exists. No decomposition is required for 2NF.

4) Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

AIM:

To apply Third Normal Form (3NF) decomposition to the given relation by removing transitive dependencies.

GIVEN RELATION:

```
mysql> CREATE TABLE EMPLOYEE (
    ->     EmpID INT PRIMARY KEY,
    ->     EmpName VARCHAR(30),
    ->     DeptID VARCHAR(10),
    ->     DeptName VARCHAR(30),
    ->     ManagerID VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO EMPLOYEE VALUES
    -> (101, 'Arun',  'D01', 'Research', 'M201'),
    -> (102, 'Meena', 'D01', 'Research', 'M201'),
    -> (103, 'Kiran', 'D02', 'Robotics', 'M202'),
    -> (104, 'Ravi',  'D03', 'AI Lab',  'M203');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM EMPLOYEE;
+-------+---------+--------+----------+-----------+
| EmpID | EmpName | DeptID | DeptName | ManagerID |
+-------+---------+--------+----------+-----------+
|   101 | Arun    | D01    | Research | M201      |
|   102 | Meena   | D01    | Research | M201      |
|   103 | Kiran   | D02    | Robotics | M202      |
|   104 | Ravi    | D03    | AI Lab   | M203      |
+-------+---------+--------+----------+-----------+
4 rows in set (0.00 sec)
```

FUNCTIONAL DEPENDENCIES

- EmpID → EmpName, DeptID
- DeptID → DeptName, ManagerID

Here, the transitive dependency is:

- EmpID → DeptID → DeptName, ManagerID

DeptName and ManagerID depend on EmpID **through DeptID**, creating a transitive dependency.

DECOMPOSE INTO 3NF

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE EMPLOYEE_3NF (
    ->     EmpID INT PRIMARY KEY,
    ->     EmpName VARCHAR(30),
    ->     DeptID VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE DEPARTMENT (
    ->     DeptID VARCHAR(10) PRIMARY KEY,
    ->     DeptName VARCHAR(30),
    ->     ManagerID VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO EMPLOYEE_3NF VALUES
    -> (101, 'Arun', 'D01'),
    -> (102, 'Meena', 'D01'),
    -> (103, 'Kiran', 'D02'),
    -> (104, 'Ravi', 'D03');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO DEPARTMENT VALUES
    -> ('D01', 'Research', 'M201'),
    -> ('D02', 'Robotics', 'M202'),
    -> ('D03', 'AI Lab', 'M203');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM EMPLOYEE_3NF;
+-------+---------+--------+
| EmpID | EmpName | DeptID |
+-------+---------+--------+
|   101 | Arun    | D01    |
|   102 | Meena   | D01    |
|   103 | Kiran   | D02    |
|   104 | Ravi    | D03    |
+-------+---------+--------+
4 rows in set (0.00 sec)

mysql>
```

```
mysql> SELECT * FROM DEPARTMENT;
+--------+----------+-----------+
| DeptID | DeptName | ManagerID |
+--------+----------+-----------+
| D01    | Research | M201      |
| D02    | Robotics | M202      |
| D03    | AI Lab   | M203      |
+--------+----------+-----------+
3 rows in set (0.00 sec)
```

RESULT:

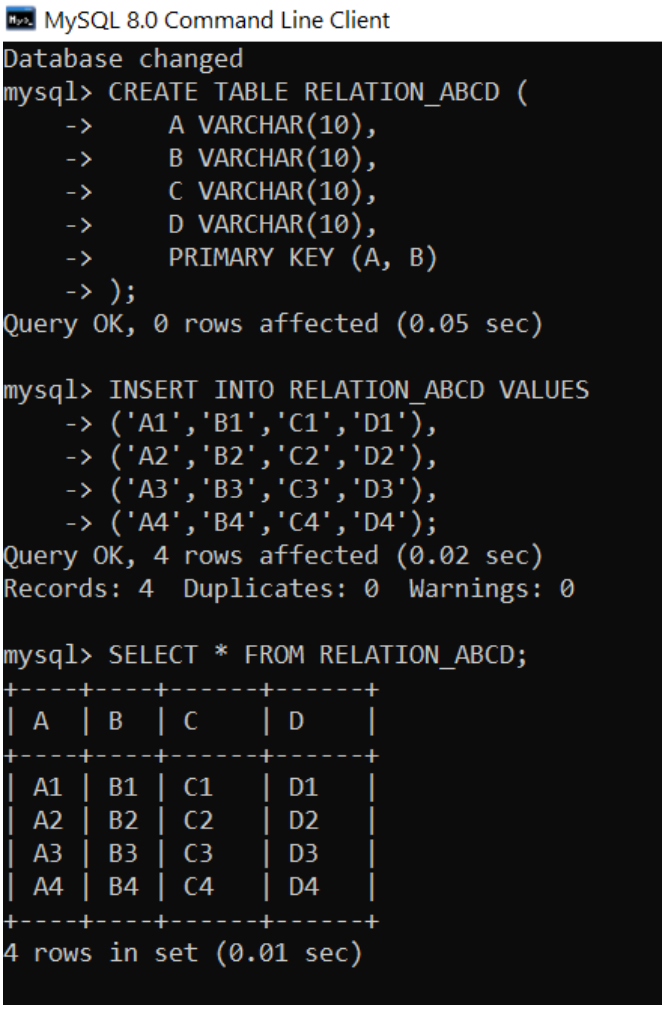
Thus, the EMPLOYEE relation was successfully decomposed into 3NF by removing the transitive dependency and separating department details into the DEPARTMENT relation.

5) Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.

AIM:

To verify whether the given relation is in Boyce-Codd Normal Form (BCNF) and decompose it into BCNF if required.

GIVEN RELATION:



FUNCTIONAL DEPENDENCIES:

- AB → C
- C → D
- D → A

VERIFY CANDIDATE KEY AB

```
mysql> SELECT A, B, COUNT(*) AS Record_Count
    -> FROM RELATION_ABCD
    -> GROUP BY A, B;
+----+----+--------------+
| A  | B  | Record_Count |
+----+----+--------------+
| A1 | B1 |            1 |
| A2 | B2 |            1 |
| A3 | B3 |            1 |
| A4 | B4 |            1 |
+----+----+--------------+
4 rows in set (0.01 sec)

mysql>
```

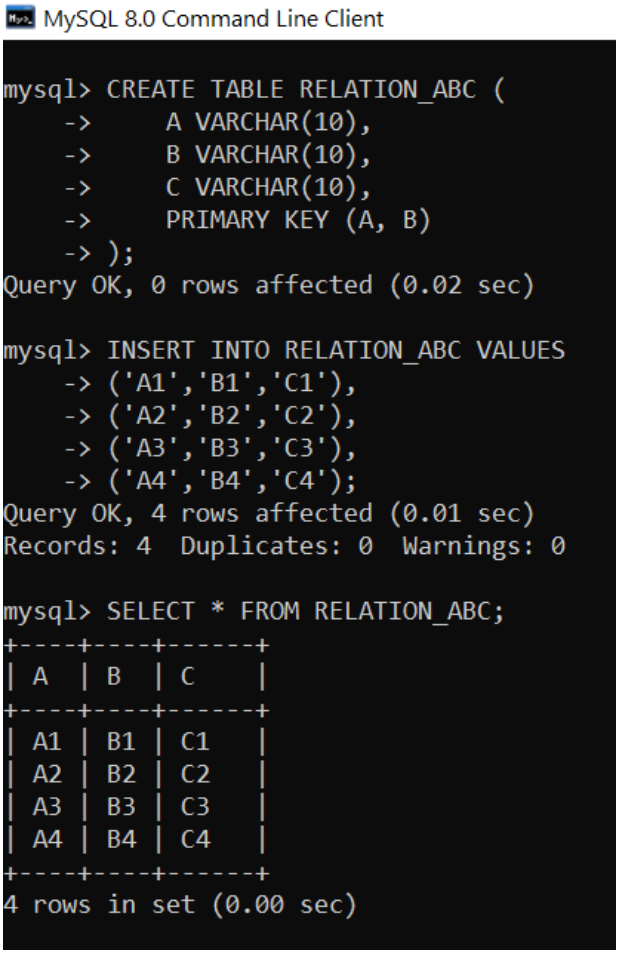
Therefore, AB is a candidate key.

CREATE FIRST BCNF DECOMPOSITION:

```
mysql> CREATE TABLE RELATION_CD (
    ->     C VARCHAR(10) PRIMARY KEY,
    ->     D VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RELATION_CD VALUES
    -> ('C1','D1'),
    -> ('C2','D2'),
    -> ('C3','D3'),
    -> ('C4','D4');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RELATION_CD;
+----+------+
| C  | D    |
+----+------+
| C1 | D1   |
| C2 | D2   |
| C3 | D3   |
| C4 | D4   |
+----+------+
4 rows in set (0.00 sec)
```



From:

- C → D

Decompose into:

- R1(C,D)
- R2(A,B,C)

SECOND BCNF DECOMPOSITION:

```
mysql> CREATE TABLE RELATION_CA (
    ->     C VARCHAR(10) PRIMARY KEY,
    ->     A VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE RELATION_BC (
    ->     B VARCHAR(10),
    ->     C VARCHAR(10),
    ->     PRIMARY KEY (B, C)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO RELATION_CA VALUES
    -> ('C1','A1'),
    -> ('C2','A2'),
    -> ('C3','A3'),
    -> ('C4','A4');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO RELATION_BC VALUES
    -> ('B1','C1'),
    -> ('B2','C2'),
    -> ('B3','C3'),
    -> ('B4','C4');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

```
mysql> SELECT * FROM RELATION_CA;
+----+------+
| C  | A    |
+----+------+
| C1 | A1   |
| C2 | A2   |
| C3 | A3   |
| C4 | A4   |
+----+------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM RELATION_BC;
+----+----+
| B  | C  |
+----+----+
| B1 | C1 |
| B2 | C2 |
| B3 | C3 |
| B4 | C4 |
+----+----+
4 rows in set (0.00 sec)
```

Final Decomposition:

- R1(C,D)
- R2(C,A)
- R3(B,C)

RESULT:

The relation R(A,B,C,D) is not in BCNF. It is successfully decomposed into: (C,D), (C,A), and (B,C).

6) Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

AIM:

To experimentally verify whether a given decomposition is lossless using the tabular (Chase) algorithm.

GIVEN RELATION:

```
mysql> CREATE TABLE RESEARCH_PROJECT (
    ->     ResearcherID VARCHAR(10),
    ->     ProjectID VARCHAR(10),
    ->     ProjectName VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO RESEARCH_PROJECT VALUES
    -> ('R101','P101','AI Research'),
    -> ('R102','P102','Robotics'),
    -> ('R103','P103','Cyber Security'),
    -> ('R104','P104','Cloud Computing');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM RESEARCH_PROJECT;
+--------------+-----------+-----------------+
| ResearcherID | ProjectID | ProjectName     |
+--------------+-----------+-----------------+
| R101         | P101      | AI Research     |
| R102         | P102      | Robotics        |
| R103         | P103      | Cyber Security  |
| R104         | P104      | Cloud Computing |
+--------------+-----------+-----------------+
4 rows in set (0.00 sec)
```

CREATE DECOMPOSED RELATIONS

```
mysql> CREATE TABLE RESEARCHER_PROJECT (
    ->     ResearcherID VARCHAR(10),
    ->     ProjectID VARCHAR(10)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> CREATE TABLE PROJECT_DETAILS (
    ->     ProjectID VARCHAR(10),
    ->     ProjectName VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO RESEARCHER_PROJECT VALUES
    -> ('R101','P101'),
    -> ('R102','P102'),
    -> ('R103','P103'),
    -> ('R104','P104');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO PROJECT_DETAILS VALUES
    -> ('P101','AI Research'),
    -> ('P102','Robotics'),
    -> ('P103','Cyber Security'),
    -> ('P104','Cloud Computing');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

CHASE TABLE:

Initial Chase Table:

Row	ResearcherID	ProjectID	ProjectName
R1	a1	a2	b3
R2	b1	a2	a3

FD: ProjectID → ProjectName

Since both rows have the same ProjectID (a2),

ProjectName takes the common distinguished value (a3).

Row	ResearcherID	ProjectID	ProjectName
R1	a1	a2	a3
R2	b1	a2	a3

A row contains all distinguished symbols.

Therefore, the decomposition is LOSSLESS.

VERIFY USING SQL JOIN:

```
mysql> SELECT r.ResearcherID,
    ->        r.ProjectID,
    ->        p.ProjectName
    -> FROM RESEARCHER_PROJECT r
    -> JOIN PROJECT_DETAILS p
    -> ON r.ProjectID = p.ProjectID;
+--------------+-----------+-----------------+
| ResearcherID | ProjectID | ProjectName     |
+--------------+-----------+-----------------+
| R101         | P101      | AI Research     |
| R102         | P102      | Robotics        |
| R103         | P103      | Cyber Security  |
| R104         | P104      | Cloud Computing |
+--------------+-----------+-----------------+
4 rows in set (0.01 sec)

mysql>
```

RESULT:

Thus, using the Chase algorithm, the decomposition

- R1(ResearcherID, ProjectID)
- R2(ProjectID, ProjectName)

was experimentally verified to be lossless-join, since the chase table produces a row containing all distinguished symbols and the join reconstructs the original relation.

7) Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

AIM:

To identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose the relation into Fourth Normal Form (4NF).

GIVEN RELATION:

MySQL 8.0 Command Line Client

```
mysql> CREATE TABLE TEACHER (
    ->     TID VARCHAR(10),
    ->     Subject VARCHAR(30),
    ->     Hobby VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO TEACHER VALUES
    -> ('T01','DBMS','Cricket'),
    -> ('T01','DBMS','Music'),
    -> ('T01','DSA','Cricket'),
    -> ('T01','DSA','Music'),
    -> ('T02','Networks','Reading'),
    -> ('T02','AI','Reading');
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM TEACHER;
+------+----------+---------+
| TID  | Subject  | Hobby   |
+------+----------+---------+
| T01  | DBMS     | Cricket |
| T01  | DBMS     | Music   |
| T01  | DSA      | Cricket |
| T01  | DSA      | Music   |
| T02  | Networks | Reading |
| T02  | AI       | Reading |
+------+----------+---------+
6 rows in set (0.00 sec)
```

MULTI-VALUED DEPENDENCIES

- TID →→ Subject
- TID →→ Hobby

DECOMPOSE INTO 4NF:

```
mysql> CREATE TABLE TEACHER_SUBJECT (
    ->     TID VARCHAR(10),
    ->     Subject VARCHAR(30),
    ->     PRIMARY KEY (TID, Subject)
    -> );
Query OK, 0 rows affected (0.02 sec)

mysql> INSERT INTO TEACHER_SUBJECT VALUES
    -> ('T01','DBMS'),
    -> ('T01','DSA'),
    -> ('T02','Networks'),
    -> ('T02','AI');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM TEACHER_SUBJECT;
+-----+----------+
| TID | Subject  |
+-----+----------+
| T01 | DBMS     |
| T01 | DSA      |
| T02 | AI       |
| T02 | Networks |
+-----+----------+
4 rows in set (0.00 sec)
```

```
mysql> CREATE TABLE TEACHER_HOBBY (
    ->     TID VARCHAR(10),
    ->     Hobby VARCHAR(30),
    ->     PRIMARY KEY (TID, Hobby)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> INSERT INTO TEACHER_HOBBY VALUES
    -> ('T01','Cricket'),
    -> ('T01','Music'),
    -> ('T02','Reading');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM TEACHER_HOBBY;
+-----+---------+
| TID | Hobby   |
+-----+---------+
| T01 | Cricket |
| T01 | Music   |
| T02 | Reading |
+-----+---------+
3 rows in set (0.00 sec)
```

- TID →→ Subject
- TID →→ Hobby

Therefore, the original relation contains independent multi-valued dependencies.

Decomposition:

- TEACHER_SUBJECT(TID, Subject)
- TEACHER_HOBBY(TID, Hobby)

RESULT:

Thus, the multi-valued dependencies were identified and the TEACHER relation was successfully decomposed into 4NF.

8) Demonstrate how join dependencies lead to redundancy.Decompose a given relation to 5NF (Project-Join Normal Form).

AIM:

To identify the functional dependencies and candidate key of the given relation.

GIVEN RELATION:

QUERY 1 – Verify SID → SName

Therefore:

- SID → SName

QUERY 2 – Verify CID → CName, Instructor

Therefore:

- CID → CName
- CID → Instructor

QUERY 3 – Verify Candidate Key (SID, CID)

Every (SID, CID) combination uniquely identifies one record.

Therefore:

- (SID, CID) → Grade
- Candidate Key = (SID, CID)

FUNCTIONAL DEPENDENCIES:

- SID → SName
- CID → CName
- CID → Instructor

- (SID, CID) → Grade

CANDIDATE KEY:

- (SID, CID)

RESULT:

Thus, the functional dependencies were identified and **(SID, CID)** was determined as the candidate key of the STUDENT_COURSE relation successfully.

9) Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.

AIM:

To identify the functional dependencies and candidate key of the given relation.

GIVEN RELATION:

QUERY 1 – Verify SID → SName

Therefore:

- SID → SName

QUERY 2 – Verify CID → CName, Instructor

Therefore:

- CID → CName
- CID → Instructor

QUERY 3 – Verify Candidate Key (SID, CID)

Every (SID, CID) combination uniquely identifies one record.

Therefore:

- (SID, CID) → Grade
- Candidate Key = (SID, CID)

FUNCTIONAL DEPENDENCIES:

- SID → SName
- CID → CName
- CID → Instructor
- (SID, CID) → Grade

CANDIDATE KEY:

- (SID, CID)

RESULT:

Thus, the functional dependencies were identified and **(SID, CID)** was determined as the candidate key of the STUDENT_COURSE relation successfully.

10) Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.

AIM:

To identify the functional dependencies and candidate key of the given relation.

GIVEN RELATION:

QUERY 1 – Verify SID → SName

Therefore:

- SID → SName

QUERY 2 – Verify CID → CName, Instructor

Therefore:

- CID → CName
- CID → Instructor

QUERY 3 – Verify Candidate Key (SID, CID)

Every (SID, CID) combination uniquely identifies one record.

Therefore:

- (SID, CID) → Grade
- Candidate Key = (SID, CID)

FUNCTIONAL DEPENDENCIES:

- SID → SName
- CID → CName
- CID → Instructor
- (SID, CID) → Grade

CANDIDATE KEY:

- (SID, CID)

RESULT:

Thus, the functional dependencies were identified and **(SID, CID)** was determined as the candidate key of the STUDENT_COURSE relation successfully.